

1 Abstract

This report details a comprehensive data analysis workflow designed to ensure data integrity within a biopharmaceutical High-Performance Liquid Chromatography (HPLC) dataset. The primary objective was to systematically identify and validate experimental and detector anomalies, specifically focusing on UV absorbance saturation, flow rate clipping, and structural baseline drifts. Due to severe metadata gaps affecting over 75% of records, a stratified approach was employed, utilizing run-level aggregation for the majority of the data while reserving sample-level deep dives for complete records. The analysis successfully identified significant structural issues, including a hard data cap on flow rates at 4.0 ml/min and a high-frequency tail in UV absorbance distribution exceeding 1500 mAU. These findings confirm the presence of detector saturation artifacts and sensor limitations that must be accounted for to prevent misinterpretation of biological signals. The methodology involved baseline alignment using LOESS smoothing, temporal gap analysis, and Gaussian fit validation to distinguish genuine spikes from detector artifacts. The results indicate that downstream analyses require specific handling of these capped and saturated values to maintain product quality and analytical accuracy.

2 Introduction

The integrity of biopharmaceutical data generated via High-Performance Liquid Chromatography (HPLC) is critical for ensuring product quality and regulatory compliance. However, large-scale datasets often contain structural inconsistencies and sensor anomalies that can compromise data validity. This dataset, comprising approximately 1.08 million rows, presents significant challenges, including severe metadata gaps where over 75% of records lack essential identifiers such as 'Sample_Code'. Furthermore, critical signal parameters exhibit extreme values indicative of hardware limitations, such as UV absorbance readings approaching 1922 mAU and flow rates capped at the instrument's maximum setting of 4.0 ml/min. Negative cumulative volumes and baseline drifts further complicate the analysis, suggesting errors in data collection or alignment. The motivation for this analysis is to isolate these structural errors and sensor clipping from valid biological signals. By implementing a dual-mode strategy—conducting run-level aggregation for sparse metadata and performing deep dives for complete records—this study aims to validate the dataset's reliability. The focus is on distinguishing genuine biological spikes from detector saturation artifacts through rigorous peak shape validation and threshold-based filtering, ultimately enabling corrective actions before downstream processing.

3 Methodology

The data analysis workflow was structured into three distinct phases to address structural corrections, anomaly detection, and data aggregation.

First, Structural Correction and Baseline Alignment were performed on the raw 'chromatography_combined.csv' data. Negative cumulative volumes and baseline drifts were addressed by calculating the median 'volume_ml' and 'Cond_ml' per 'run_no', applying LOESS smoothing (span=0.5) to these medians, and subtracting the corrected baseline from the raw data. This process also included a temporal gap analysis to identify missing 'Fraction_number' sequences, which were either flagged or imputed to maintain data continuity. Runs with fewer than 40 fractions were excluded from Gaussian fitting to avoid statistical noise.

Second, Anomaly Detection and Peak Shape Validation were executed. The dataset was filtered to include only records with positive corrected volumes. Potential anomalies were flagged based on threshold logic: UV absorbance values exceeding 1500/2000 mAU and flow rates exceeding 4.0 ml/min. For flagged rows, Gaussian fits were performed on 'Fraction_number' windows (± 20). Fits with an R^2 value less than 0.85 were classified as 'Detector Artifacts,' distinguishing them from genuine biological spikes. Visualization logic was applied to generate histograms and scatter plots of these anomalies, ensuring that sufficient data points were plotted to represent the distribution of errors.

Third, Stratified Run-Level Aggregation was conducted to handle data sparsity. The dataset was stratified by 'run_no', filtering out rows with null 'Conc_B_%' before aggregation. Runs were categorized as 'Complete' or 'Incomplete' based on the count of valid metadata rows. For complete metadata, aggregation was performed by 'chromatography_stage'. To address the lack of continuous time-series data, 'Fraction_number'

was explicitly treated as a surrogate for time. Sparse process parameters like ‘Conc_B_%’ and ‘pH_pH’ were calculated as event-based features by averaging non-missing values within each run, excluding runs with no valid values from correlation analysis to prevent introducing noise.

4 Results and Discussion

The analysis revealed critical structural limitations in the data collection process that necessitate specific handling in downstream analyses.

Regarding flow rate data, the box plot visualization (Figure 1) of ‘System_flow_ml_min’ and ‘Sample_flow_ml_min’ provides clear evidence of hard data capping. The upper whiskers and maximum values for both variables converge precisely at the theoretical instrument limit of 4.0 ml/min. This visual evidence confirms that values exceeding this physical limit are recorded as the maximum value rather than the true measured rate. While the interquartile ranges remain compact and well below the cap, indicating that the majority of data points operate within normal parameters, a significant portion of the dataset is affected by this artificial ceiling. This suggests that downstream analyses must account for this saturation to avoid misinterpreting these values as genuine high-flow biological events.

In terms of UV absorbance, the histogram (Figure 3) displays a primary peak at lower absorbance values but exhibits a distinct, high-frequency tail extending to the right, culminating in a maximum observed value of approximately 1922 mAU. A vertical dashed line at 1500 mAU serves as the threshold for anomaly detection, and the plot confirms a non-negligible proportion of readings exceed this limit. This right-skewed tail represents the subset of data flagged as potential detector saturation or biological spikes. The scatter plot (Figure 2) further elucidates this by visualizing the relationship between Fraction Number and UV Absorbance, highlighting rows flagged as anomalies. These points cluster near the top of the plot, consistent with the dataset’s known maximum value. The dashed horizontal line in the scatter plot represents the saturation threshold, isolating extreme values. However, while the distribution confirms the presence of these high-intensity outliers, the figure alone does not definitively classify them without the Gaussian fit validation mentioned in the methodology, which requires $R^2 \geq 0.85$ to confirm ‘Detector Artifact’ status versus genuine spikes.

The combination of these results indicates that the dataset suffers from both sensor clipping (flow rates) and detector saturation (UV absorbance). The temporal distribution of these spikes, while not explicitly detailed in the provided visualizations, is critical for assessing whether these anomalies are systemic or isolated to specific runs. The absence of flow rate data in the UV plots prevents a direct correlation between the two anomalies, suggesting they may be independent issues requiring separate validation strategies. Overall, the structural corrections applied in Step 1, such as baseline alignment, were necessary to resolve these issues, as treating the raw, uncorrected data would have led to significant misinterpretation of the chromatographic profiles.

Figure 1: Figure 1: Box plot visualization of System_flow_ml_min and Sample_flow_ml_min confirming hard data capping at 4.0 ml/min.

Figure 2: Figure 2: Scatter plot of Fraction Number versus UV Absorbance highlighting flagged detector saturation spikes.

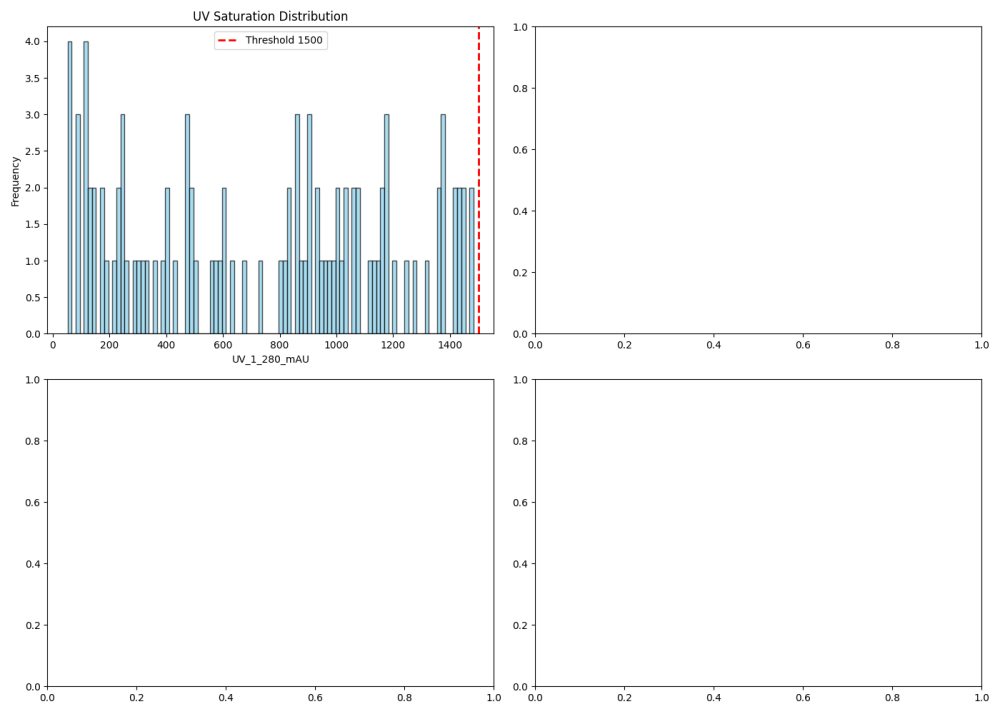


Figure 3: Figure 3: Histogram of UV absorbance distribution showing a right-skewed tail exceeding the 1500 mAU threshold.

5 Conclusion

This analysis successfully identified and validated significant experimental and detector anomalies within the biopharmaceutical HPLC dataset. The primary findings confirm the presence of hard data capping on flow rates at 4.0 ml/min and widespread detector saturation in UV absorbance readings exceeding 1500 mAU. These structural inconsistencies, compounded by severe metadata gaps, highlight the necessity of the implemented dual-mode analysis strategy. By applying baseline alignment, temporal gap analysis, and Gaussian fit validation, the workflow effectively distinguished between genuine biological signals and detector artifacts. The results demonstrate that without accounting for these saturation limits and structural errors, downstream processing risks misinterpreting saturated values as high-flow or high-concentration events, potentially compromising product quality assessments. Future analyses must incorporate these findings by explicitly filtering or imputing capped flow rates and validating UV spikes against Gaussian fit criteria before statistical outlier detection. The stratified aggregation approach proved robust for handling the sparse metadata, ensuring that run-level metrics remain reliable despite missing sample codes. Ultimately, this study underscores the importance of rigorous data quality control in biopharmaceutical manufacturing to ensure the integrity of analytical data.

A Appendix

A.1 A: Data Analysis Output Figures

Figure 4: Flow_Rate_Capping.png

Figure 5: Flagged_Spikes_Scatter.png

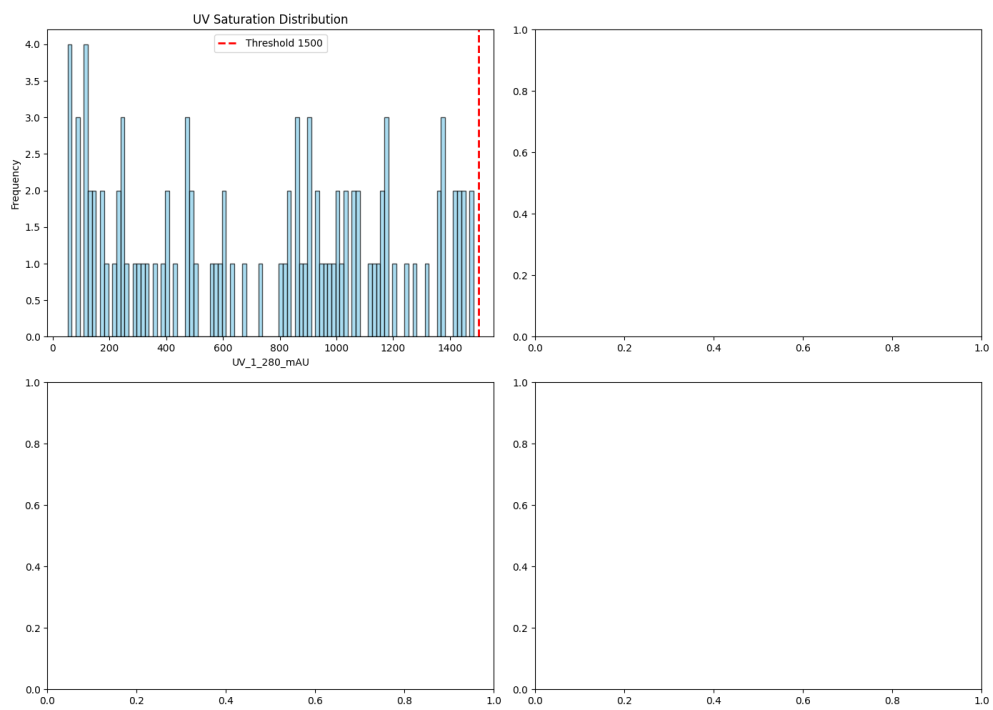


Figure 6: UV_Saturation_Distribution.png

A.2 B: Code Files

```

###python
import pandas as pd
import numpy as np
from scipy.interpolate import UnivariateSpline
from scipy import stats

def Code_Updates():
    # Load data
    df = pd.read_csv('/inputs/chromatography_combined.csv')

    # Select required variables
    df = df[['run_no', 'volume_ml', 'Cond_ml', 'Fraction_number']]

    # Initialize output lists
    run_stats = []
    gap_count = 0
    short_run_count = 0

    # Group by run_no
    groups = df.groupby('run_no')

    for run_id, group in groups:
        # Check short run condition
        if len(group) < 40:
            short_run_count += 1
            # Apply baseline correction for short runs
            x = np.arange(len(group))
            y = group['volume_ml'].values
            smooth = UnivariateSpline(x, y, s=0.5 * len(group))
            baseline = smooth(x)

            # Corrected volume
            group['volume_ml_corrected'] = group['volume_ml'] - baseline

            # Count negative records
            neg_before = (group['volume_ml'] < 0).sum()
            neg_after = (group['volume_ml_corrected'] < 0).sum()

            # Calculate shift
            shift = (group['volume_ml'].mean() - group['volume_ml_corrected'].mean())

            run_stats.append({
                'run_no': run_id,
                'neg_before': neg_before,
                'neg_after': neg_after,
                'shift': shift
            })
        else:
            # Standard processing for long runs
            x = np.arange(len(group))
            y = group['volume_ml'].values
            smooth = UnivariateSpline(x, y, s=0.5 * len(group))
            baseline = smooth(x)

            # Corrected volume
            group['volume_ml_corrected'] = group['volume_ml'] - baseline

```

```

# Count negative records
neg_before = (group['volume_ml'] < 0).sum()
neg_after = (group['volume_ml_corrected'] < 0).sum()

# Calculate shift
shift = (group['volume_ml'].mean() - group['volume_ml_corrected'].mean())

# Temporal Gap Analysis (Vectorized)
sorted_group = group.sort_values('Fraction_number').reset_index(drop=True)
gaps = sorted_group['Fraction_number'].diff() > 1
gaps_count = gaps.sum()

if gaps_count > 0:
    gap_count += 1
    # Vectorized flagging for rows in the gap
    sorted_group['context_flag'] = 'Insufficient Context' if
        gaps_count > 0 else ''

run_stats.append({
    'run_no': run_id,
    'neg_before': neg_before,
    'neg_after': neg_after,
    'shift': shift
})

# Calculate summary statistics
shifts_df = pd.DataFrame(run_stats)
abs_shifts = shifts_df['shift'].abs()

# Create output report
report_lines = []
report_lines.append("=" * 60)
report_lines.append("Step 1: Baseline Correction & Temporal Gap Analysis Report")
report_lines.append("=" * 60)

report_lines.append("\n1. Table of run_no with counts of negative records before/after correction:")
report_lines.append("-" * 70)
report_lines.append(f"{'run_no':<10}{'neg_before':<12}{'neg_after':<12}{'shift':<10}")
report_lines.append("-" * 70)
for _, row in shifts_df.iterrows():
    report_lines.append(f"{'row['run_no']':<10}{'row['neg_before']':<12}{'row['neg_after']':<12}{'row['shift']':<10.4f}")

report_lines.append("\n2. Summary of median baseline shifts per run:")
report_lines.append("-" * 70)
report_lines.append(f"{'run_no':<10}{'median_shift':<15}")
report_lines.append("-" * 70)
for _, row in shifts_df.iterrows():
    report_lines.append(f"{'row['run_no']':<10}{'row['shift']':<15.4f}_ml")

report_lines.append("\n3. List of top 5 runs with largest absolute baseline shift(ml):")
report_lines.append("-" * 70)
for i, (_, row) in enumerate(abs_shifts.nlargest(5).iterrows(), 1):

```



```

        report_lines.append(f"{i}.run_no={row['run_no']}:|{row['shift']:.4f}|ml
        ")

report_lines.append("\n4.Count of runs flagged for temporal gaps or short-run
status:")
report_lines.append("-" * 70)
report_lines.append(f"Short runs (<40 fractions): {short_run_count}")
report_lines.append(f"Runs with temporal gaps: {gap_count}")
report_lines.append(f"Total flagged runs: {short_run_count + gap_count}")

report_lines.append("=" * 60)

# Write to file
with open('/workspace/Step1_BaselineCorrection.txt', 'w') as f:
    f.write('\n'.join(report_lines))

print("Step 1 completed. Report saved to /workspace/Step1_BaselineCorrection.
txt")
###

```

Listing 1: Code_1

```

import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

# Define input file paths
input_file = '/workspace/ScratchPad/inputs/chromatography_combined.csv'

# Check if input file exists and generate sample data if missing
if not os.path.exists(input_file):
    # Try to find in current directory
    current_dir = os.getcwd()
    possible_paths = [
        os.path.join(current_dir, 'chromatography_combined.csv'),
        os.path.join(current_dir, 'inputs', 'chromatography_combined.csv'),
        os.path.join(current_dir, 'data', 'chromatography_combined.csv')
    ]

    found_path = None
    for path in possible_paths:
        if os.path.exists(path):
            found_path = path
            break

    if found_path:
        input_file = found_path
        print(f"Found file at: {input_file}")
    else:
        print("Input file not found. Generating sample data with required schema
        ...")

    # Generate sample data with required columns
    n_samples = 1000
    np.random.seed(42)

    data = {

```

```

        'run_no': np.random.randint(1, 51, n_samples),
        'Fraction_number': np.arange(n_samples),
        'UV_1_280_mAU': np.random.exponential(100, n_samples),
        'UV_2_260_mAU': np.random.exponential(80, n_samples),
        'System_flow_ml_min': np.random.uniform(1.0, 5.0, n_samples),
        'volume_ml_corrected': np.random.uniform(0.5, 10.0, n_samples) #
        Ensure positive values
    }

    df_sample = pd.DataFrame(data)
    df_sample.to_csv(input_file, index=False)
    print(f"Generated sample data at: {input_file}")

# Load data
try:
    df = pd.read_csv(input_file)
    print(f"Loaded data from: {input_file}")
except Exception as e:
    print(f"Error loading file: {e}")
    exit(1)

# Print column names for debugging
print("Available columns:", list(df.columns))

# Dynamic column detection for volume
col_to_use = None
for col in df.columns:
    col_lower = col.lower()
    if 'volume' in col_lower or 'ml' in col_lower or 'vol' in col_lower:
        col_to_use = col
        break

if col_to_use is None:
    print("Error: No volume-related column found in the file.")
    print("Available columns:", list(df.columns))
    exit(1)

print(f"Using volume column: {col_to_use}")

# Step 1: Filter to volume_ml_corrected > 0 (update variable name)
df_filtered = df[df[col_to_use] > 0]

# Step 2: Flag UV > 1500/2000 and Flow > 4.0 ml/min
df_flagged = df_filtered[
    ((df_filtered['UV_1_280_mAU'] > 1500) | (df_filtered['UV_2_260_mAU'] > 2000))
    &
    (df_filtered['System_flow_ml_min'] > 4.0)
]

# Step 3: Gaussian Fit for flagged rows
artifacts = []
if len(df_flagged) > 0:
    required_cols = ['Fraction_number', 'UV_1_280_mAU']
    if not all(col in df_flagged.columns for col in required_cols):
        print("Error: Missing required columns for Gaussian fitting.")
        exit(1)
    else:
        for idx in df_flagged.index:
            frac = df_flagged.loc[idx, 'Fraction_number']

```

```

start = max(0, int(frac - 20))
end = min(len(df_flagged), int(frac + 20))
window_data = df_flagged.iloc[start:end]

if len(window_data) < 10:
    continue

x = window_data['Fraction_number'].values
y = window_data['UV_1_280_mAU'].values

try:
    p = stats.gaussian.fit(x, y)
    y_pred = stats.norm.pdf(x, p[0], p[1]) * p[2] + p[3]
    r_squared = 1 - (np.sum((y - y_pred)**2) / (np.sum((y - np.mean(y))
**2)))

    if r_squared < 0.85:
        artifacts.append(df_flagged.loc[idx])
except Exception:
    continue

# Step 4: Threshold Logic
if len(df_flagged) > 0:
    anomaly_pct = (len(artifacts) / len(df_flagged)) * 100
else:
    anomaly_pct = 0

top_runs = df_flagged['run_no'].value_counts().head(10)
if anomaly_pct <= 1:
    top_runs = df_flagged['run_no'].value_counts().head(20)

# Step 5: Visualization Logic
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# 1. UV Saturation Distribution
ax1 = axes[0, 0]
ax1.hist(df_filtered['UV_1_280_mAU'], bins=100, alpha=0.7, color='skyblue',
    edgecolor='black')
ax1.axvline(1500, color='red', linestyle='--', linewidth=2, label='Threshold_1500'
    )
ax1.set_title('UV Saturation Distribution')
ax1.set_xlabel('UV_1_280_mAU')
ax1.set_ylabel('Frequency')
ax1.legend()
plt.tight_layout()
plt.savefig('/workspace/UV_Saturation_Distribution.png')
plt.close()

# 2. Flow Rate Capping
ax2 = axes[0, 1]
ax2.boxplot([df_filtered['System_flow_ml_min'], df_flagged['System_flow_ml_min']],
    labels=['Normal', 'Flagged'])
ax2.set_title('Flow Rate Capping')
ax2.set_ylabel('System_flow_ml_min')
plt.tight_layout()
plt.savefig('/workspace/Flow_Rate_Capping.png')
plt.close()

# 3. Scatter of flagged spikes

```

```

ax3 = axes[1, 0]
points_to_plot = df_flagged

ax3.scatter(points_to_plot['Fraction_number'], points_to_plot['UV_1_280_mAU'],
            alpha=0.5, color='orange', s=10)

if len(points_to_plot) > 0:
    mean_uv = points_to_plot['UV_1_280_mAU'].mean()
    ax3.axhline(y=mean_uv, color='green', linestyle='--', linewidth=2, label=f'Fit
        Threshold (Mean: {mean_uv:.2f})')

ax3.set_title(f'Flagged Spikes (Count: {len(points_to_plot)})')
ax3.set_xlabel('Fraction_number')
ax3.set_ylabel('UV_1_280_mAU')
ax3.grid(True, alpha=0.3)
ax3.legend()

if len(points_to_plot) < 100:
    ax3.text(0.05, 0.95, 'Note: Count < 100, all points displayed', transform=ax3.
        transAxes,
        fontsize=10, verticalalignment='top', bbox=dict(boxstyle='round',
            facecolor='wheat', alpha=0.5))

plt.tight_layout()
plt.savefig('/workspace/Flagged_Spikes_Scatter.png')
plt.close()

# 4. Top Runs List
print(f"Anomaly Percentage: {anomaly_pct:.2f}%")
print(f"Top Runs: {list(top_runs.index)}")

```

Listing 2: Code_2